

PersianLang

زبان برنامه نویسی فارسی

IDE کامل با پشتیبانی راست به چپ — ساخته شده با Python و PyQt5





درباره پروژه

About the Project

هدف

ایجاد یک زبان برنامه‌نویسی با کلیدواژه‌های فارسی تا یادگیری برنامه‌نویسی به زبان مادری آسان‌تر شود

ابزارها

Python · PyQt5 · QSyntaxHighlighter · PptxGenJS

نتیجه

IDE کامل با ادیتور RTL، هایلایت کد، اجرای برنامه فارسی، مدیریت فایل و Undo/Redo

قابلیت‌های PersianLang



ادیتور RTL

پشتیبانی کامل راست‌به‌چپ با PyQt5 و
— QTextOption کرسر و انتخاب درست



هایلایت کد

رنگ‌بندی کلیدواژه، کامنت و رشته با
QRegExp و QSyntaxHighlighter



مدیریت متغیر

تعریف و نمایش متغیرهای عددی و رشته‌ای
در self.variables (dict)



شرط‌ها

اگر / وگرنه — پشتیبانی از >, <, =, >=, <=
برای عدد و رشته



توابع و تکرار

تعریف تابع فارسی، اجرا با 'اجرا'، تکرار با
شمارنده دلخواه



Undo / Redo

Ctrl+Z/Y یا دکمه — تاریخچه کامل حفظ
می‌شود بدون تداخل block-format



چالش‌ها و مشکلات

۱

tkinter در RTL

tkinter پشتیبانی بومی راست‌به‌چپ ندارد — متن فارسی وارونه نمایش داده می‌شود و cursor در جهت اشتباه حرکت می‌کند

✓ راه‌حل:

انتقال کامل به PyQt5 یا QPlainTextEdit و
`setLayoutDirection(Qt.RightToLeft)`

۲

خراب Undo / Redo

هر بار که کاربر تایپ می‌کند، block-format پاراگراف تغییر می‌کند و این تغییر به عنوان یک عملیات جداگانه در تاریخچه ثبت می‌شود

✓ راه‌حل:

جداسازی Ctrl+Z/Y با `matches(QKeySequence.Undo)` از فراخوانی `super()`

۳

مقایسه رشته در شرط

کاربران به‌طور طبیعی = می‌نوشتند اما interpreter فقط == می‌شناخت — خروجی همیشه نادرست بود و مشکل پیدا کردنش سخت بود

✓ راه‌حل:

افزافه کردن `elif '=' in condition` با اولویت پایین‌تر از == و سایر عملگرها



سه لایه اصلی — معماری کد

لایه `PersianLang (QMainWindow)` • `RTLOutput` • `RTLEditor` — `UI`

ویجت‌های `PyQt5` مسئول نمایش ادیتور `RTL`، پنل خروجی و نوار دکمه‌ها هستند `RTLEditor`. با `override` کردن `keyPressEvent` جهت پاراگراف را در هر تایپ حفظ می‌کند.

لایه `PersianHighlighter (QSyntaxHighlighter)` — `Syntax`

با `QRegExp` الگوهای کلیدواژه، کامنت (`///#`) و رشته (`""`) را تشخیص می‌دهد و با `QTextCharFormat` رنگ‌گذاری می‌کند. هر بار که `document` تغییر کند خودکار اجرا می‌شود.

لایه `run()` • `execute()` • `get_value()` • `normalize()` — `Interpreter`

`run()` حلقه اصلی است — خط به خط می‌خواند و ساختارهای کنترلی را پردازش می‌کند `execute()`. دستورات تکی را اجرا می‌کند. `normalize()` و `get_value()` داده را آماده می‌سازند.

تابع پایه normalize(self, text)

```
def normalize(self, text)
```

```
# Remove Unicode zero-width chars
text = text.replace("\u200c", "")
text = text.replace("\u200f", "")
text = text.replace("\u200e", "")

# Normalize Arabic Ya/Ka -> Persian
text = text.replace("ي", "ی")
text = text.replace("ك", "ک")

# Convert Persian digits ۰-۹
text = text.translate(
    str.maketrans(
        "۰۱۲۳۴۵۶۷۸۹",
        "0123456789"
    )
)

return text.strip()
```

چرا normalize لازم است؟

وقتی کاربر فارسی تایپ می‌کند، کاراکترهای نامرئی مثل ZWNJ حروف قرار می‌گیرند. اگر حذف نشوند، مقایسه رشته‌ها ناموفق می‌شود.

نرمال‌سازی ی / ک

در بعضی keyboard layout ها ی و ک به شکل عربی تایپ می‌شوند. بدون این تبدیل، یک کلمه ممکن است با دو encoding متفاوت نوشته شود و interpreter آن را نشناسد.

اعداد فارسی به لاتین

اعداد ۰ تا ۹ کدپوینت یونیکد متفاوتی از 0-9 لاتین دارند. با translate() یک mapping خطی همه را تبدیل می‌کند تا int() و float() بتوانند پردازش کنند.

تابع پایه `get_value(self, text)`

```
def get_value(self, text)
```

```
# Step 1: normalize first
text = self.normalize(text)

# Step 2: variable name?
if text in self.variables:
    return self.variables[text]

# Step 3: integer?
if text.isdigit():
    return int(text)

# Step 4: float?
try:
    return float(text)
except: pass

# Step 5: quoted string?
if text.startswith(''):
    return text[1:-1]

# Step 6: return as raw text
return text
```

چرا `get_value` مهم است؟

هر بار که interpreter مقداری می‌خواند — چه از متغیر، چه از جمع، چه از شرط — از این تابع استفاده می‌کند. همه تبدیل نوع یکجا انجام می‌شود.

اولویت متغیر (Step 2)

اگر متن در `self.variables` باشد مقدار ذخیره شده برمی‌گردد. یعنی می‌توان نام متغیر را مستقیم نوشت: جمع سن و ' — 10 سن ' به مقدارش تبدیل می‌شود.

زنجیره تشخیص نوع

به ترتیب امتحان می‌شود: عدد صحیح ← اعشاری ← رشته داخل گیومه ← متن خام. اولین تطابق برمی‌گردد و بقیه اجرا نمی‌شوند.

PersianHighlighter (QSyntaxHighlighter)



```
class PersianHighlighter(QSyntaxHighlighter)
```

```
# Rules defined in __init__:
fmt = QTextCharFormat()
fmt.setForeground(QColor("blue"))
fmt.setFontWeight(700)

self.rules.append((
    QRegExp(r"\bمتغیر"), fmt))
self.rules.append((
    QRegExp(r"#(?:^\n)*"), comment_fmt))

# Called automatically per paragraph:
def highlightBlock(self, text):
    for pattern, fmt in self.rules:
        idx = pattern.indexIn(text)
        while idx >= 0:
            length = pattern.matchedLength()
            self.setFormat(idx, length, fmt)
            idx = pattern.indexIn(text, idx+length)
```

QSyntaxHighlighter چیست؟

کلاس پایه PyQt5 برای رنگ‌آمیزی کد. فقط باید از آن ارث‌بری کنی و `highlightBlock()` را `override` کنی — PyQt5 خودش آن را برای هر پاراگراف صدا می‌زند.

QRegExp + rules list

هر قانون یک `tuple` از الگو، فرمت (است). الگوهای کلیدواژه‌ها، کامنت‌ها (`#//`) و رشته‌های داخل گیومه جداگانه در `__init__` تعریف می‌شوند.

setFormat(idx, length, fmt)

این متد فقط رنگ را روی آن محدوده اعمال می‌کند بدون اینکه محتوای سند تغییر کند — کاملاً ایمن و بهینه برای `re-render`های مکرر.

RTLEditor (QPlainTextEdit)



class RTLEditor(QPlainTextEdit)

```
# Setup RTL in __init__:
self.setLayoutDirection(Qt.RightToLeft)

opt = QTextOption()
opt.setTextDirection(Qt.RightToLeft)
opt.setAlignment(Qt.AlignRight)
self.document().setDefaultTextOption(opt)

self.setFont(QFont("Tahoma", 12))

# Override keyPressEvent:
def keyPressEvent(self, e):
    # Intercept Undo/Redo BEFORE super()
    if e.matches(QKeySequence.Undo):
        self.undo(); return
    if e.matches(QKeySequence.Redo):
        self.redo(); return

    super().keyPressEvent(e)
    # Then apply RTL block-format to new paragraph
```

setLayoutDirection RTL

کل وِجت RTL می‌شود cursor — از راست شروع می‌کند، انتخاب از راست به چپ کشیده می‌شود و scroll bar هم جابجا می‌شود.

QTextOption برای پاراگراف

setDefaultTextOption() پیش‌فرض هر پاراگراف جدید را راست‌چین و RTL می‌کند. بدون این، خطوط جدید ممکن است LTR باشند.

Undo/Redo پیش از super()

مشکل اصلی: اگر Ctrl+Z پس از super() گرفته شود، تغییر block-format هم در تاریخچه ثبت می‌شود و با هر Undo یک عملیات اضافه برگشت می‌کند.

کد کلیدی — execute(): پردازش دستورات تکی

```
def execute(self, line)
```

```
line = self.normalize(line)
if line.startswith("#"): return

# -- variable --
if line.startswith("متغیر"):
    name, val = temp.split("=", 1)
    self.variables[name] = get_value(val)

# -- print --
if line.startswith("چاپ"):
    self.write(get_value(text))

# -- arithmetic --
if line.startswith("جمع"):
    a, b = line.split("و", 1)
    self.write(get_value(a) + get_value(b))

# -- input --
if line.startswith("ورودی"):
    v, ok = QDialogDialog.getText(...)
    self.variables[name] = v

raise Exception(f"دستور ناشناخته {line}")
```

اول normalize

هر خط قبل از هر بررسی normalize می‌شود — کاراکتر زیرنویس، ی/ک و اعداد فارسی اصلاح می‌شوند

کامنت بدون خطا

خطوط شروع با # یا // بلافاصله با return نادیده گرفته می‌شوند

split('=', 1)

فقط اولین = جداکننده است — مقدار رشته مثل نام="علی=خوب" هم درست کار می‌کند

عملیات با 'و'

جمع/تفریق/ضرب/تقسیم با 'و' دو طرف را جدا می‌کنند و get_value() می‌کنند

ورودی با QDialogDialog

dialog فارسی باز می‌شود؛ مقدار تایپ‌شده مستقیم در self.variables ذخیره می‌شود

Exception آخر

اگر هیچ دستوری تطبیق نداشت خطای فارسی پرتاب می‌شود — run() آن را نمایش می‌دهد



کد کلیدی — run(): حلقه اصلی interpreter

```
def run(self)
```

```
self.variables.clear()
self.functions.clear()
lines = editor.toPlainText().splitlines()
i = 0
while i < len(lines):
    line = normalize(lines[i])
    try:
        if line.startswith("تابع"):
            body = []; i += 1
            while line != "پایان تابع":
                body.append(lines[i]); i += 1
            self.functions[name] = body

        elif line.startswith("تکرار"):
            count = int(get_value(...))
            for _ in range(count):
                for cmd in block: execute(cmd)

        elif line.startswith("اگر"):
            # Evaluate condition
            ...

    else: execute(line)
except Exception as e:
    self.write(f"خطا: {e}")
i += 1
```

پاک سازی اولیه

variables و functions در شروع هر اجرا پاک می‌شوند تا اجراهای متوالی تداخل نداشته باشند

while با شمارنده i

به جای for، از while و اندستی استفاده می‌شود تا بلاک‌های چندخطی یکجا خوانده شوند

جمع‌آوری — تابع

خطوط بدنه تا 'پایان تابع' در list جمع می‌شوند و در self.functions[name] ذخیره می‌شوند

بلاک — تکرار

تعداد از get_value() گرفته و بدنه تا 'پایان' جمع‌آوری و بار اجرا می‌شود

شرط بندی — اگر

به ترتیب >, <=, ==, >, <, = چک می‌شوند. بلاک true یا false اجرا می‌شود

try / except

هر دستور محافظت شده — خطا به فارسی در خروجی نمایش داده می‌شود و برنامه crash نمی‌کند

تعریف و فراخوانی — تابع و اجرا: قابلیت

تعریف تابع در کد فارسی

```
# Define function
تابع خوشامد
چاپ به PersianLang خوش آمدید
چاپ برنامه‌نویسی به فارسی
پایان تابع

# Call function
اجرا خوشامد
```

interpreter ذخیره تابع در

```
# run() -- store function body
name = line[len("تابع"):].strip()
body = []; i += 1
while normalize(lines[i]) != "پایان تابع":
    body.append(lines[i]); i += 1
self.functions[name] = body

# Call with 'ajra' keyword
for cmd in self.functions[name]:
    self.execute(cmd)
```

تعریف ساده

تابع با کلیدواژه 'تابع' شروع و با
'پایان تابع' تمام می‌شود — هر
محتوایی می‌تواند داخل آن باشد

ذخیره به عنوان list

بدنه تابع به عنوان لیست رشته در
`self.functions[name]` ذخیره
می‌شود — اجرا نمی‌شود تا
'اجرا' خوانده شود

فراخوانی با 'اجرا'

`execute()` برای هر خط بدنه
فراخوانی می‌شود — متغیرهای
برنامه اصلی در دسترس تابع
هستند

بدون پارامتر

توابع در `PersianLang` پارامتر ندارند
— از متغیرهای سراسری برای
تبادل داده استفاده می‌شود



قابلیت :اگر / وگرنه — شرط بندی فارسی

نمونه کد شرطی

متغیر سن = 20
متغیر نام = "مسیح"

اگر سن <= 18
چاپ بزرگسال
وگرنه
چاپ کودک
پایان

اگر نام = "مسیح"
چاپ سلام مسیح
پایان

run() پردازش شرط در

```
condition = line[len("اگر"):].strip()
true_block = []; false_block = []
current = true_block; i += 1
while normalize(lines[i]) != "پایان":
    if normalize(lines[i]) == "وگرنه":
        current = false_block
    else: current.append(lines[i])
    i += 1
```

Evaluate condition by priority:

```
if ">=" in condition: ... elif "<=" in condition: ...
elif "==" in condition: ... elif ">" in condition: ...
elif "<" in condition: ... elif "=" in condition: ...
```

جمع آوری دو بلاک

خطوط true_block و false_block
جداگانه جمع آوری می شوند —
'وگرنه' جهت را تغییر می دهد

ترتیب اولویت مهم است

ابتدا <= و > چک می شوند،
سپس ==، بعد < و > آخر =
تنها — تا تداخل علائم نباشد

== مثل =

کاربران فارسی به طور طبیعی =
می نویسند interpreter — هر دو
را قبول می کند

رشته و عدد

مقایسه هم برای اعداد و هم برای
رشته ها کار می کند —
get_value() نوع را تشخیص
می دهد

قابلیت :تکرار و کمک

کد فارسی -دستور تکرار

```
# Loop with fixed count
تکرار 3
چاپ سلام دنیا
پایان

# Loop with variable
متغیر n = 5
تکرار n
چاپ خط شماره
پایان
```

run()پردازش تکرار در

```
count = int(get_value(
    line[len("تکرار"):].strip()))
block = []; i += 1
while normalize(lines[i]) != "پایان":
    block.append(lines[i])
    i += 1

for _ in range(count):
    for cmd in block:
        self.execute(cmd)
```

عدد یا متغیر

تعداد تکرار می‌تواند عدد مستقیم یا نام متغیر باشد —
آن را تبدیل می‌کند

جمع‌آوری بدنه



دستور کمک

با تایپ 'کمک' در ادیتور و اجرای برنامه، راهنمای کامل تمام دستورات با
مثال در پنل خروجی نمایش داده می‌شود. `execute()` یک `handler`
اختصاصی برای این دستور دارد که ۳۰ خط راهنما را به ترتیب `write()`
می‌کند.

برنامه کامل — نمونه کد فارسی

Define variable

سن = 25
نام = "مسیح"

Condition with string compare

نام = "مسیح"
چاپ سلام مسیح عزیز
وگرنه
چاپ کاربر ناشناس
پایان

Loop

3 تکرار
چاپ برنامه نویسی فارسی
پایان

Function

تابع خوشامد
خوش آمدید PersianLang چاپ به
بایان تابع

اجرا خوشامد
نمایش متغیرها

خروجی برنامه

✓ سن 25 =

✓ نام = مسیح

سلام مسیح عزیز

برنامه نویسی فارسی

برنامه نویسی فارسی

برنامه نویسی فارسی

به PersianLang خوش آمدید

-----متغیرها-----

سن | 25 = نام = مسیح

= ☹ به تنهایی مثل == کار می کند



جمع‌بندی

انتقال موفق ادیتور از QtKinter به PyQt5 با RTL کامل

پیاده‌سازی QSyntaxHighlighter با QRegExp برای هایلایت فارسی

رفع مشکل Undo/Redo با جداسازی QKeySequence قبل از super()

پشتیبانی = و == در شرطها — رشته و عدد هر دو مقایسه می‌شوند

توابع، شرط‌بندی، تکرار و دستور کمک کامل پیاده‌سازی شده‌اند

— run() · execute() · get_value() · normalize() چهار ستون اصلی





با تشکر از همراهی شما

تهیه کننده: مسیح درستکار